INTA: Intent-Based Translation for Network Configuration with LLM Agents

Yunze Wei[†], Xiaohui Xie[†*], Tianshuo Hu[†], Yiwei Zuo[‡], Xinyi Chen[†], Kaiwen Chi[†], Yong Cui[†*]
[†]Department of Computer Science and Technology, Tsinghua University, Beijing, China
[‡]Australian National University, Canberra, Australia

2025 IEEE 33rd International Conference on Network Protocols (ICNP) | 979-8-3315-0376-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICNP65844.2025.11192391

***Abstract*—Translating configurations between different network devices is a common yet challenging task in modern network operations. This challenge arises in typical scenarios such as replacing obsolete hardware and adapting configurations to emerging paradigms like Software Defined Networking (SDN) and Network Function Virtualization (NFV). Engineers need to thoroughly understand both source and target configuration models, which requires considerable effort due to the complexity and evolving nature of these specifications. To promote automation in network configuration translation, we propose INTA, an intent-based translation framework that leverages Large Language Model (LLM) agents. The key idea of INTA is to use *configuration intent* as an intermediate representation for translation. It first employs LLMs to decompose configuration files and extract fine-grained intents for each configuration fragment. These intents are then used to retrieve relevant manuals of the target device. Guided by a syntax checker, INTA incrementally generates target configurations. The translated configurations are further verified and refined for semantic consistency. We implement INTA and evaluate it on real-world configuration datasets from the industry. Our approach outperforms state-of-the-art methods in translation accuracy and exhibits strong generalizability. INTA achieves an accuracy of 98.15% in terms of both syntactic and view correctness, and a command recall rate of 84.72% for the target configuration. The semantic consistency report of the translated configuration further demonstrates its practical value in real-world network operations.**

***Index Terms*—Network Configuration, Configuration Translation, Large Language Model, Network Management, Network Operation**

I. Introduction

Configuration translation has become an increasingly critical task in modern network operations and maintenance. As networks evolve, outdated or faulty devices are often replaced with more advanced and efficient alternatives [14], [28], making configuration translation essential to ensure continuity and compatibility. The adoption of Software Defined Networking (SDN) and Network Function Virtualization (NFV) further drives the need to integrate traditional network devices into SDN architectures or migrate their configurations to NFV environments [7], [13]. This often entails translating traditional command line interface (CLI) configurations into SDN controller-based or NFV-oriented representations.

This work was supported by NSFC Project under Grant 62132009 and Grant 62221003.

* Corresponding Authors: Xiaohui Xie and Yong Cui.

979-8-3315-0376-5/25/$31.00 ©2025 IEEE

Translating configurations across different network platforms is a complex and challenging task. This paper takes cross-vendor CLI configuration translation as a representative example to illustrate the inherent difficulties of configuration translation. Network engineers need to interpret the functionality and intent of complex source device configurations accurately [4] and translate them into semantically equivalent configurations for target devices. The vendor-specific nature of CLI syntax further complicates this process, requiring deep expertise in the configuration models of multiple vendor devices. This requirement imposes substantial training costs on network engineers, as they must acquire in-depth knowledge of diverse vendor-specific configurations. Such expertise is time-consuming to develop and difficult to maintain as network technologies and device models continue to evolve.

Both industry and academia are actively exploring automated methods for configuration translation. NAssim [7] constructs device configuration models and uses NetBERT to recommend target commands. ConfigTrans [28] takes a step further by combining heuristic rules with Large Language Models (LLMs) to translate diverse types of commands. However, both approaches still require much manual work and struggle to generalize across diverse scenarios. General programming language translation methods [34], [38] also fall short of meeting the requirements for this task due to the diversity of network device configuration syntax and the scarcity of configuration corpora. We summarize the challenges of automated configuration translation into three core aspects: (1) correctly understanding the logic and intent of the source configurations, (2) accurately retrieving and interpreting the target device manuals, and (3) generating configurations that are both syntactically correct and semantically consistent.

The rapid development of LLMs has brought novel opportunities for automated network configuration translation. Recent successful applications of LLM-based multi-agent systems [15], [36] have demonstrated their potential for task understanding and solution generation. In this paper, we propose INTA, an Intent-based framework for Network configuration Translation with LLM Agents. Inspired by prior research on intent-driven approaches [3], [23], INTA introduces intent as an intermediate representation that bridges the gap between cross-vendor device configurations. INTA comprises four key components: a *parser* that parses the source configuration and extracts corresponding command manuals, an *intent-based manual retriever* that analyzes configuration intents

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

and retrieves relevant target-vendor device manuals through a two-stage retrieval and voting mechanism, a *syntax-guided incremental translator* that incrementally generates the target configuration with syntactic guidance, and an *LLM-based semantic verifier* that checks and refines the translation to ensure semantic consistency. INTA ultimately produces a target-device-specific configuration and a detailed report evaluating its syntactic correctness and semantic consistency, thereby assisting network engineers in verification and deployment.

We implement INTA and evaluate it on a real-world configuration dataset collected from industry sources. In the router configuration translation scenario from Nokia to Huawei, INTA achieved 98.15% accuracy on view and syntactic correctness, and a command-level match rate of 84.72% compared to reference translations. We also conduct ablation studies on multiple components of INTA to validate their effectiveness. To provide deeper insight into INTA's workflow, we present a representative case study that demonstrates the end-to-end processing of a complete example. In addition, we validated INTA in a switch scenario from Cisco to Huawei, achieving a syntactic correctness of 96.50%. These results demonstrate the effectiveness and generalizability of INTA across different migration scenarios. In addition, we manually verify the accuracy of the semantic consistency report, revealing INTA's practical value.

The main contributions of this paper are as follows:

1) We analyze the difficulties and challenges in cross-vendor network configuration translation, and propose INTA, an intent-based configuration translation framework leveraging LLM agents to address these challenges.
2) We design an intent-based target device manual retrieval module to retrieve target device manuals accurately.
3) We develop a syntax-guided incremental translation module and a semantic refinement module to enhance the syntactic correctness and semantic consistency.
4) We implement INTA and evaluate it on real-world datasets across different network migration scenarios.

II. Background and Motivation

A. Network Device Configuration

Network device configuration is an essential part of network operation, covering the entire life cycle of network devices, including setup, maintenance, and troubleshooting.

There are several ways to configure network devices. The most traditional and widely used method is the CLI, which requires administrators to write device configuration files and then manually input the configuration commands or automatically import these configuration files for deployment. The NETCONF [11] protocol and the YANG [6], [35] language are designed for advanced network device configuration and are widely used in SDN for data center or campus network operation. Although new protocols provide more convenient possibilities for network configuration, the CLI is still indispensable in various scenarios, such as device initialization. Given its complexity and widespread use, we focus on CLI configuration translation in this paper.

Nokia
```
config
    port 1/3/14
        ethernet
            lldp
                dest-mac nearest-non-tpmr
                    tx-tlvs port-desc sys-name sys-desc sys-cap
```

Huawei
```
system-view
    interface GigabitEthernet1/3/14
        lldp tlv-enable basic-tlv port-description
        lldp tlv-enable basic-tlv system-name
        lldp tlv-enable basic-tlv system-description
        lldp tlv-enable basic-tlv system-capability
```

(a) One-to-many mapping.

Nokia
```
config
    router Base
        policy-options
            prefix-list "SYSTEM"
                prefix xx.x.xx.x/32
```

Huawei
```
system-view
    ip ip-prefix SYSTEM permit xx.x.xx.x 32
```

(b) View depth differences.

Nokia
```
config
    router Base
        interface "IF_NAME"
            port 1/3/14:101
            bfd 100 receive 100 multiplier 3
        ospf 0
            area 0.0.0.x
                interface "IF_NAME"
                    bfd-enable
```

Huawei
```
system-view
    ospf 100 router-id xx.x.xx.xx
        area 0.0.0.x
    interface GigabitEthernet1/3/14
        description "IF_NAME"
        ospf enable 100 area 0.0.0.x
        ospf bfd enable
        ospf bfd min-rx-interval 100 min-tx-interval 100 detect-multiplier 3
```

(c) Device design logic differences.

Fig. 1. Design differences in different configuration models.

A CLI command on a network device typically comprises *keywords*, *parameters*, and the *view* or operational context in which the command is executed. A single CLI command line usually consists of *keywords, parameters*, and many mandatory/optional items. The following is an example of a command template for the Huawei NE40E Router [22].

```
ip address <ip-address>
  { <mask> | <mask-length> } [ <sub> ]
```

In this template, `ip address` is the keyword, `<ip-address>` is the mandatory parameter, `{ <mask> | <mask-length> }` means that one of the parameters is required, and `[ <sub> ]` means that the parameter is optional.

A CLI often includes multiple *views*, each containing a set of specific commands. For example, the command mentioned above may appear in the interface view, the Mtunnel view, and the ACL address pool view. Both the functionality and the parameters of the command can vary across different views.

B. Configuration Translation

Network configuration translation is the process of converting a source device's configuration into that of a target

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

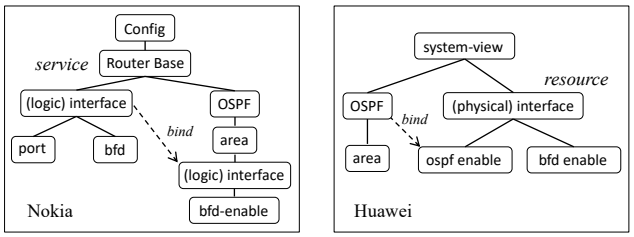


Fig. 2. Configuration logic graph for Fig. 1(c).

device while preserving consistent behavior. This is often required when replacing devices from one vendor with another, typically driven by functionality upgrades, disaster recovery, policy changes, or cost considerations.

The general process of configuration translation includes the following four steps: (1) Understanding the intent and functionality of the source device configuration; (2) Consulting device manuals to translate the source configuration into the corresponding target configuration; (3) Analyzing the syntax and semantic correctness of the translated configuration; (4) Applying the translated configuration to the target device and monitoring the behavior of the target device. This process is complex and demands considerable expert knowledge and experience. Since the configuration models of different vendors vary significantly, translation requires experts who are familiar with both vendors' systems. We summarize the key differences in configuration models across vendors as follows.

Design differences in configuration models. The difficulties of configuration translation stem from *the significant design differences in different vendors' configuration models*, which can lead to substantial variations in configuration commands. We summarize the differences in device configuration models into the following three aspects: one-to-many mappings, differences in view depth, and differences in design logic. We use the Huawei NE40E router and the Nokia 7750SR router as examples to illustrate these differences.

① *One-to-many mapping* is the simplest form of configuration differences. A typical case occurs when a single command in Nokia's device corresponds to multiple commands in Huawei's device, as shown in Fig. 1(a).

② *Differences in view depth* are the most apparent structural differences. As illustrated in Fig. 1(b), configuring an IP prefix list in Huawei requires only a single command under the `system-view`, whereas in Nokia, the same task involves navigating through four levels of views.

③ *Differences in design logic* are the fundamental reason why configuration translation is challenging. Vendors adopt distinct design principles for their configuration models, leading to substantial differences in configuration logic. For example, Fig. 1(c) demonstrates the configurations of OSPF and BFD protocols on Huawei and Nokia devices. The configuration logic graphs are shown in Fig. 2. Nokia adopts a *service-centric* approach. It binds a pre-defined logical interface in the OSPF instance and enables the BFD protocol. In contrast, Huawei adopts a *resource-centric* approach. It first creates an OSPF instance and then enables it in a physical interface, along with enabling the BFD protocol. Accurately translating these configurations requires a deep understanding of the underlying design logic of each vendor's device model.

TABLE I
COMPARISON WITH EXISTING METHODS.

Method	End-to-end Translation	Logic Difference Understanding	Migration Overhead
NAssim [7]	✗	✗	Low
ConfigTrans [28]	✓	✗	High
INTA	✓	✓	Low

Related work: mapping-based methods. The most straightforward approach to configuration translation is directly mapping lines of configuration commands from one to another. NAssim [7] constructs the device configuration model as a semantics-enhanced tree structure and uses NetBERT (fine-tuned SBERT [32]) to map the nodes of Vendor Device Model (VDM) and Unified Device Model (UDM) used in SDN. However, NAssim is not an end-to-end solution. It still relies on human selection from the recommended commands. ConfigTrans [28] goes further based on NAssim's VDM models to achieve end-to-end cross-vendor translation. It uses a heuristic method to translate commands with parameters, while commands without parameters are translated with the help of LLMs. However, ConfigTrans still relies on pre-defined rules and pre-built parameter correspondence tables, which require substantial manual work. Its heuristic algorithm also exhibits limited generalization across different configuration scenarios.

In contrast to mapping-based methods, INTA uses an intent-based method to bridge the gap between different configuration models. As shown in Table I, INTA not only achieves end-to-end translation but also captures the underlying logical differences between configuration models, while maintaining low migration overhead across scenarios.

C. Challenges and Opportunities

Based on an analysis of configuration translation methods in the industry, we identify three key challenges in the translation process and explore how LLM-based approaches offer promising opportunities to address them.

C1: Interpreting the logic and intent of source configurations. A fundamental challenge in configuration translation is accurately interpreting the logic and intent behind the source configuration commands. While functional descriptions of individual commands can typically be found in command manuals, synthesizing these descriptions into a coherent, high-level intent representation remains difficult.

C2: Retrieving relevant target device manuals. Generating an equivalent configuration requires identifying which commands on the target device can fulfill the intent, as well as determining their exact syntax (e.g., keywords, parameters, hierarchical structure). This necessitates retrieving relevant information from the target device's manuals, which is challenging due to the large volume of the manual corpus and the often vague or abstract nature of the configuration intent.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

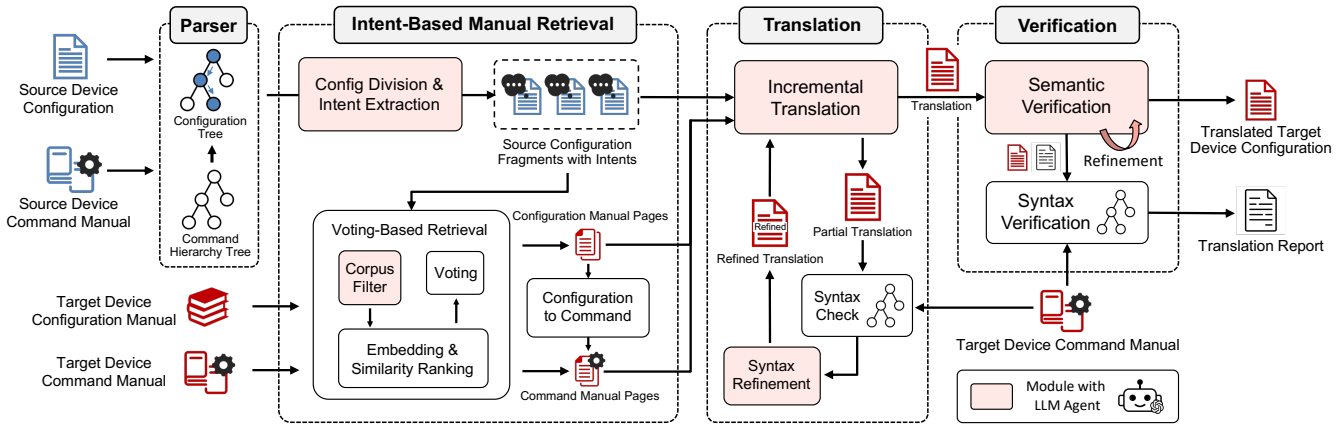


Fig. 3. System workflow of INTA.

C3: Generating syntactically valid and semantically consistent configurations. The translated configuration must strictly adhere to the target device's complex syntax rules while preserving the original semantics of the source configuration. Achieving both syntactic correctness and semantic equivalence is non-trivial and requires carefully designed mechanisms.

Opportunities of LLM-based approaches. The significant advances in LLMs' understanding and reasoning abilities [17] open new possibilities for configuration translation. Specifically, our proposed method, INTA, uses LLMs to analyze configuration intent in place of human engineers (C1), assist in the retrieval of relevant manual content (C2), and perform incremental translation under the guidance of syntax checkers, followed by semantic verification and refinement (C3).

III. Design of INTA

A. System Overview

The workflow of INTA is shown in Fig. 3, which consists of four main components: the configuration *parser*, the intent-based *retrieval* module for target device manuals, the configuration *translation* module, and the *verification* module.

Workflow. The source device configuration to be translated is first parsed by the configuration *parser*. The main component of the parser is a command hierarchy tree constructed from the command manuals of the source device and VDM, similar to NAssim [7]. It parses the syntax of each line of configuration and maps the line on the command hierarchy tree, obtaining each command's view structure and corresponding manuals. The details of the configuration parser are shown in Appendix A. The parsed configuration commands (together with the corresponding command manual pages) then enter the *intent-based retrieval* module, which first divides the configuration into fragments and analyzes the intents of each fragment. Then the configuration fragments with intents are used to retrieve the target device manuals, as detailed in Section III-B. The *translation* module uses the retrieved configuration and the target device manual pages to translate the source configuration fragments. This is an incremental process where each fragment is translated based on the preceding translation. We also perform syntax checks and refinements to enhance this process. Finally, the *verification* module makes semantic verification and refinement, eliminating redundant information and supplementing missing semantic details. The final output consists of the translated configuration and an accompanying translation report documenting syntax correctness and semantic consistency. The overall workflow employs multiple LLM agents to address simpler, well-defined subtasks, leading to improved performance over end-to-end approaches.

Two types of device manuals. We use two types of manuals. *Configuration manuals* describe the procedures to implement specific functions, indicating which commands to use. *Command manuals* provide comprehensive syntax definitions and functional descriptions for all the commands, which also serve as the basis for constructing command hierarchy trees. Examples of both manuals are shown in Appendix B.

B. Intent-Based Target Manuals Retrieval

The retrieval of target device configuration and command manuals is essential for high-quality translation, as general LLMs lack domain-specific knowledge and thus require external manual injection. However, accurately retrieving these manuals is a challenging task. Inspired by existing works [3], [23], we leverage configuration intent to bridge the significant gap between configuration and manuals. In this process, intent serves as a vendor-neutral abstraction layer for manual retrieval and configuration translation, which is better suited for LLMs to understand and process. To the best of our knowledge, we are the first to use "intent" as an intermediate representation for configuration translation. We first use LLM to split the source configuration into fragments based on functionality and extract the intent of each fragment. Then we retrieve the corresponding manual pages of the target device based on the intent. We further use the retrieved configuration manuals to enhance the retrieval of command manuals.

1) Configuration Intent Extraction: We employ LLM to split the source configuration into fragments, and extract the intent of each fragment. The fragments are divided by LLMs based on functionality units inferred from the semantics of the source configuration and its corresponding command manual

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

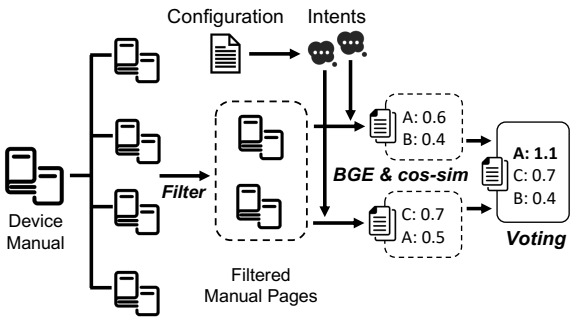


Fig. 4. Detailed workflow of voting-based retrieval.

pages. To produce stylistically consistent and structured intent descriptions that support reliable manual retrieval, we adopt the In-Context Learning (ICL) method [5], [9], using templates and examples to guide the LLM's intent extraction process. We also ask the LLM to extract intents at different levels: a *general* description of the entire configuration fragment and *detailed* descriptions of each sub-module in the fragment. This helps to improve the recall rate of the subsequent manual retrieval step. The prompt template includes output requirements, few-shot examples, configuration to divide, corresponding source device manuals, etc., as shown in Fig. 10 (Appendix C).

2) Voting-Based Target Manual Retrieval: The detailed workflow of the proposed retrieval pipeline is shown in Fig. 4, which consists of three main steps: manual corpus filtering, manual context embedding, and voting mechanism.

Manual corpus filtering. The configuration and command manuals of network devices are massive. For instance, the Huawei NE40E router has a configuration manual of ∼8000 pages [21] and a command manual of ∼14000 pages [20], making accurate retrieval particularly challenging. To address this, an initial filtering stage is essential to narrow the corpus, improving retrieval accuracy and efficiency. Considering the distinct characteristics of configuration and command manuals, we adopt two separate filtering strategies.

① *Filter for configuration manual corpus.* The configuration manual is organized as a hierarchical directory tree, with each directory level clearly described using natural language. Inspired by the way humans navigate and comprehend manuals, we propose to employ LLMs to interpret the directory and perform effective manual filtering. We provide the LLM with the source configuration fragment, its corresponding manual content, and the directory structure of the target device's configuration manual, allowing LLM to select the most relevant directory. However, high-level directory entries are often too coarse-grained (e.g., `IP Service` and `IP Routing`), making it difficult for the LLM to determine the appropriate directory accurately. Therefore, we concatenate the first and second-level directories together, allowing LLM to obtain information from both levels simultaneously, enabling it to make more accurate selections. The prompt template for LLM manual filtering is shown in Fig. 12 (Appendix C), which includes the task and guidelines, source configuration with manuals, target configuration manual list, etc.

② *Filter for command manual corpus.* We use BM25 [33], a classic probabilistic ranking function that scores documents based on term frequency and document length normalization, to perform initial filtering on the command manual. Experiments show that using BM25 and LLM-filter on command manuals yields similar performance (Section IV-C), but BM25 is more cost-efficient and faster.

Manual context embedding. Manual pages typically include brief descriptions that outline their functionality. We aim to match the extracted intents with these descriptions to identify the corresponding manual pages. We use the BGE model [8] based on BERT to encode intent sentences and manual contexts. To disambiguate commands that may appear in multiple views, we augment each context with its manual file path, which provides view-specific information and helps filter out irrelevant views. To sum up, the manual context we use includes: the title of a manual page, the page description, and the manual file path. We also include CLI commands in the context for retrieving command manual pages. Finally, we compute the cosine similarity between the intent description and each manual context embedding. The top-k manuals with the highest scores are selected as retrieval results for the corresponding intent.

Voting mechanism. The voting mechanism is a crucial strategy for enhancing the recall rate of relevant target device manuals. After the preceding processing steps, we obtain configuration fragments, their corresponding intent descriptions (both at the fragment level and for each individual command), and a top-k list of manuals retrieved for each intent. To further improve the relevance of the retrieval results, we adopt a voting-based integration approach. Specifically, for each intent description, we aggregate the retrieved manual lists using a weighted voting scheme, where each manual's similarity score serves as its voting weight. The final score of each manual is calculated as the cumulative sum of its scores across all individual retrieved manual lists. The manuals are then re-ranked based on their aggregated scores, and the resulting list serves as the knowledge base for the subsequent configuration translation stage. This method can effectively improve the manual recall rate (Section IV-C).

3) Configuration-to-Command Manual Retrieval: To further enhance the retrieval performance of command manuals, we adopt a cross-retrieval approach. Specifically, we leverage command references in the previously retrieved configuration manual pages to supplement and refine the command manual retrieval. We use Algorithm 1 to map configuration manual pages to command manual pages. The input to the algorithm is a dictionary mapping each configuration manual page m to its retrieval score s. The output is a set of command manuals retrieved based on these configuration manuals. The algorithm iterates over each configuration manual page $m \in M$, and uses the rule-based `extract_configuration_commands` method to extract the configuration commands referenced by m (line 3). For each extracted command $c \in C$, it then applies the `get_command_manual` method, which leverages an auto-

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Configuration-to-Command Retrieval

Data: Retrieved configuration manual pages with scores $M = \{m_i \mapsto s_i\}$
Result: Aggregated command manual pages with scores $M' = \{m'_j \mapsto s'_j\}$

1 $M' \leftarrow \{\}$;
2 **foreach** $m \mapsto s \in M$ **do**
3 $\quad C \leftarrow$ extract_configuration_commands(m)
4 $\quad$ **foreach** $c \in C$ **do**
5 $\quad\quad m' \leftarrow$ get_command_manual(c)
6 $\quad\quad M'[m'] \leftarrow M'[m'] + s$; `// Default to 0 if` $m' \notin M'$
7 $\quad$ **end**
8 **end**
9 **return** M'

Algorithm 2: Semantic Verification and Refinement

Data: Source configuration s, Translated configuration t, Source device manuals M_s, Target device manuals M_t
Result: Refined configuration t', Semantic verification report r_1

1 $r_0 \leftarrow$ LLM_semantic_analysis(s, t)
2 $t' \leftarrow t$;
3 **foreach** $(s_i, t_i, \textit{is_consistent}, cmt) \in r_0$ **do**
4 $\quad$ **if** *is_consistent* $=$ *False* **then**
5 $\quad\quad m_s \leftarrow$ retrieve_relevant_manuals(s_i, M_s);
6 $\quad\quad m_t \leftarrow$ retrieve_relevant_manuals(t_i, M_t);
7 $\quad\quad t'' \leftarrow$ LLM_semantic_refinement$(s, t, s_i, t_i, m_s, m_t, cmt)$
8 $\quad\quad$ **if** *syntax_errors*$(t'') \leq$ *syntax_errors*(t') **then**
9 $\quad\quad\quad t' \leftarrow t''$
10 $\quad\quad$ **end**
11 $\quad$ **end**
12 **end**
13 $r_1 \leftarrow$ LLM_semantic_analysis(s, t')
14 **return** t', r_1

matically constructed mapping from the command manual, to retrieve the corresponding command manual page m' (line 5). The scores of all retrieved command manual pages are aggregated and stored in M' (line 6). Finally, the command manual pages obtained through this process are merged with the previously retrieved list. Their scores are then aggregated and re-ranked to yield the final set of command manual pages.

C. Syntax Checker Guided Incremental Translation

We incrementally translate configuration fragments from the source device into the target device's configuration based on the previously retrieved target device manuals, with the translation process guided by the command hierarchy tree.

1) Incremental Translation: Incremental translation refers to the sequential translation process of source configuration fragment by fragment. We construct a comprehensive prompt by combining source device configuration commands, corresponding manuals, and previously retrieved target device configuration and command manuals. Then we employ LLMs' understanding, analysis, and generation capabilities for translation. Since LLMs have context length limits [16] and excessively long contexts may degrade output quality [24], we opted for a fragment-by-fragment rather than full-text translation approach. While it is possible to translate each fragment independently, we observe that configuration fragments often exhibit strong interdependencies in practice. Therefore, we adopt an incremental translation approach that leverages the translations of preceding fragments as context, allowing forward dependencies to be preserved across fragments. The prompt template includes manual command conventions, source commands with manuals, retrieved target manuals, etc., as shown in Fig. 13 (Appendix C).

2) Syntax Checker Guidance: This enables real-time checking of both syntax and view structure, as well as providing refinement hints for the incremental translation results. Due to the hallucination phenomenon inherent in LLMs [18], the translated configuration commands may not conform to the configuration views or syntactic requirements of the target device. Therefore, we build the command syntax and hierarchy checker using the target device's command manual and the VDM [7], similar to the configuration parser module (Appendix A). We conduct two rounds of checks on the translated configuration: a view consistency check and a syntactic validity check. In the first round, the translated configuration is validated against the command hierarchy tree, which reflects both the correct view and syntax constraints. Unmatched entries from this round may result from either view inconsistencies or syntax errors. To distinguish the cause, a second round of matching is performed against the complete set of configuration commands, ignoring view constraints and focusing solely on syntax validity. Entries that remain unmatched are identified as *syntax errors*, while those that match the full command set but not the hierarchy tree are labeled as *view errors*. We annotate the two types of errors and guide the LLM to correct the translated configuration from the previous iteration through multi-round dialogue, where the translation history is recorded during the process. If these corrections lead to improved syntactic and view accuracy, the updated commands are adopted. The prompt template for multi-round syntax correction is shown in Fig. 14 (Appendix C).

D. Verification

In the final stage of configuration translation, we perform semantic consistency verification and refinement on the translated configuration. Furthermore, syntactic verification of the final translated configuration is also included. The final translation output is accompanied by a verification report on semantic consistency and syntactic correctness, which facilitates better understanding and utilization by network operators.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

1) Semantic Verification and Refinement: The purpose of semantic verification is to analyze whether the translated configuration is semantically equivalent to the original source configuration. We observe in practice that LLMs excel at analyzing cross-vendor configuration differences, so we leverage them as semantic verifiers and refiners. The process of semantic verification and refinement is illustrated in Algorithm 2. The algorithm begins with an LLM-based analysis of semantic consistency between the translated and source configurations (line 1). The output of this step is an initial semantic consistency report r_0, composed of multiple report units. Each unit contains a translated configuration fragment t_i, its corresponding source fragment s_i, a consistency flag $is_consistent$, and an explanatory comment cmt generated by the LLM. We then iterate through all units in the report. For each unit marked as inconsistent, we perform a targeted refinement of the translation. This refinement is guided by the corresponding pages from the source and target device manuals, the source and translated configuration fragments, and the LLM-generated comment (line 7). After all inconsistent fragments have been refined, we conduct a second round of semantic analysis using the LLM to generate the final semantic consistency report r_1 (line 13). The algorithm ultimately returns the semantically refined translation t' along with its corresponding consistency report. The prompt templates for semantic verification and refinement are shown in Fig. 16 and Fig. 17, respectively, within Appendix C.

2) Syntax Verification: After semantic refinement, the translated configuration undergoes a final round of syntax verification. This process employs the same syntax checker introduced in Section III-C, where the command hierarchy tree is constructed based on the target device's command manual and its corresponding VDM. The translated configuration is then validated through line-by-line matching against this hierarchy tree. Notably, syntax verification checks both view hierarchy correctness and command syntax validity. The output of syntax verification includes the mapping between each configuration line and its corresponding command template. Commands that fail to match are labeled as `Mismatch`.

3) Translation Report: The translation report provides a comprehensive evaluation of translation output, focusing on syntactic correctness and semantic consistency. Its primary goal is to assist human experts in efficiently leveraging the translation results. While INTA achieves high translation accuracy, it cannot fully ensure compliance with device-specific syntax or guarantee semantic equivalence. The report mitigates these limitations by explicitly highlighting syntactic errors and potential semantic divergences, thereby reducing the verification burden for network engineers. Since the syntax correctness report is generated by a deterministic configuration syntax checker, its findings are theoretically sound. Similarly, the semantic consistency report identifies deviations between the source and translated configurations. While LLM-generated annotations occasionally contain errors, experimental results (Section IV-E) demonstrate that the report remains reliable and practically useful.

IV. Evaluation and analysis

We implement INTA in Python with ∼3500 lines of code (LoC). For the parser, we build the configuration tree based on the open-source configuration models from NAssim [7] and implement vendor-specific parsers (Appendix A). In the intent-based manual retrieval module, we use BGE-M3 [8] as the sentence embedding model. The embedding process is accelerated by an NVIDIA GeForce RTX 3090 or A100 GPU. We use OpenAI's SDK to call several LLMs, including GPT-4o [30], Qwen-Max [2], and DeepSeek-V3 [25] (671B).[1] We use a Huawei NE40E router in the GNS3 emulator [1] and a physical Huawei CE6881 switch for translation validation. Our system has relatively low migration overhead. Supporting a new vendor only requires scraping its manuals and modifying ∼100 LoC in the vendor-specific parser, without fine-tuning. The command syntax parser is reusable across vendors.

A. Experimental Setup

Manuals and dataset. We use Nokia 7750 SR [29] and Huawei NE40E [19] routers as our source and target devices for our main experiments. The command manuals and VDM (hierarchy of command manuals) come from the open-source dataset from NAssim [7]. We scrape the configuration manual of NE40E from the Huawei website [21]. To support full-coverage configuration translation, we retrieve content from the complete set of manuals, rather than a small subset as in ConfigTrans [28]. This enables broader applicability but also poses a much greater challenge for accurate manual retrieval. Our dataset includes 1063 lines of configuration commands from 53 files, where 16 files are real configuration files from the industry, 20 files from Huawei configuration manual examples, and 17 files from Nokia configuration manual examples. The dataset covers various settings of the routers, including basic system information, interface, route policy, filter policy, BGP/IGP protocols, VPRN, etc.

Metrics. The evaluation metrics take into account both the syntactic and view correctness, as well as the similarity to the reference configuration. *(1) Tree Match:* the matching rate on the configuration command hierarchy tree, which checks both the syntactic and view correctness. *(2) Syntax Correctness:* This metric is designed to measure the pure syntax correctness because view errors caused by some commands may affect the matching of subsequent syntax-correct commands on the hierarchy tree. *(3) BLEU* [31]: a widely used metric that focuses on precision, often used to evaluate the output quality of machine translation tasks. *(4) Exact Match:* the strict matching rate, measuring the recall rate at the full command line level. *(5) Command Match:* a recall-based metric measuring whether the required commands are successfully translated, serving as a quantitative indicator of semantic correctness. Note that *Exact Match* is intuitive but limited, as correct translations may be non-unique. *Tree Match*, *Syntax Correctness*, and *Command Match* are more informative and task-relevant metrics. We use

[1]Specific model versions: gpt-4o-2024-08-06, qwen-max-2025-01-25, deepseek-v3-0324.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

TABLE II
END-TO-END EVALUATION.

Base Model	Method	Tree Match	Syntax Correctness	BLEU-2	Exact Match	Command Match
GPT-4o	Baseline	0.7041	0.8560	0.6749	0.4980	0.6748
	IRAG-only	0.8438	0.9144	0.7727	0.5852	0.8069
	IRAG+Syntax	0.9219	0.9727	0.7688	0.6418	0.8155
	INTA (Full Method)	**0.9277**	**0.9813**	**0.7880**	**0.6531**	**0.8287**
Qwen-Max	Baseline	0.6142	0.7466	0.6680	0.4900	0.6100
	IRAG-only	0.8586	0.9616	0.7143	0.5818	0.7911
	IRAG+Syntax	0.9256	0.9769	0.7367	0.6055	0.8101
	INTA (Full Method)	**0.9364**	**0.9781**	**0.7483**	**0.6179**	**0.8169**
DeepSeek-V3	Baseline	0.7315	0.8654	0.7144	0.5959	0.6944
	IRAG-only	0.8690	0.9511	0.7284	0.5902	0.8148
	IRAG+Syntax	0.9727	0.9913	0.7458	0.6102	0.8256
	INTA (Full Method)	**0.9815**	**0.9966**	**0.7654**	**0.6464**	**0.8472**

Recall Rate@Top-k to evaluate the manual retrieval module. It denotes the percentage of queries where at least one ground-truth manual appears in the top k retrieved results.

Methods and baselines. We use the LLM-only translation results as the baseline for the end-to-end experiment. INTA is our full method, which includes the Intent-based Retrieval-Augmented Generation (IRAG) process (including intent-based manual retrieval and incremental translation), followed by syntax and semantic refinement. We also evaluate two ablated versions of INTA: one with only the IRAG process (IRAG-only), and one that includes both IRAG and Syntax Refinement (IRAG + Syntax). We use GPT-4o [30], Qwen-Max [2] and DeepSeek-V3 [25] as the base models to avoid single-model bias and to ensure more convincing results. We also include ConfigTrans [28] as a baseline method and conduct separate comparisons within its supported scope.

B. End-to-End Evaluation

Results. The result of the end-to-end evaluation is shown in Table II. In the table, bold font indicates the best result within each group of base models. Underlining denotes the best result across all methods in that column. Our full method, INTA, has demonstrated a significant improvement compared to the baseline of LLM-only translation. With DeepSeek-V3 as the base model, the *syntax correctness* rate reaches 99.66% and the *tree match* (view and syntax correctness) rate reaches 98.15%, which are 13.12% and 25.00% higher than baseline, respectively. These two metrics indicate that INTA performs well in generating target device configurations with respect to both syntactic and view correctness. The *command match* rate also has a significant improvement, reaching 84.72%, which is 15.28% higher than the baseline. This metric reflects INTA's effectiveness in recalling the necessary commands for the target device in the translated configuration. In addition, our method also shows its effectiveness on GPT-4o and Qwen-Max, indicating that INTA can adapt to different base models.

Ablation study. We can see from Table II that the method with only IRAG has a great improvement in translation performance compared to the baseline. In the IRAG-only method with DeepSeek-V3 as base model, *tree match* increases by 13.75% compared to the baseline while *command match* increases by 12.04%. The results indicate that the IRAG module enhances the LLMs' ability to generate syntactically correct and semantically consistent configurations by retrieving and incorporating relevant manuals of the target device, which serve as a crucial source of domain-specific knowledge for accurate translation. Table II also shows that our modules for syntax and semantic refinement have a significant effect on improving the translation quality. With syntax refinement, the *tree match* rate increases from 86.90% to 97.27%. This indicates that syntax checks and refinements based on command hierarchy trees and LLM can significantly improve syntax and view correctness in the incremental translation process. The semantic verification and refinement module improves command match and exact match by 2.16% and 3.62% respectively (from IRAG+Syntax to INTA). It also plays a key role in generating translation quality reports tailored for human engineers, the accuracy of which is evaluated in Section IV-E.

TABLE III
COMPARISON WITH CONFIGTRANS.

Method	Acc. (w/ param.)	Acc. (w/o param.)
INTA (with GPT-4o)	0.8615	0.8957
ConfigTrans	0.8247	0.7550

Comparison with ConfigTrans. We conduct experiments on the dataset used in ConfigTrans [28], which includes BGP and OSPF commands. As shown in Table III, INTA outperforms ConfigTrans in translation accuracy for both parameterized and non-parameterized commands. More importantly, unlike ConfigTrans, INTA does not rely on mode-specific heuristic algorithms and pre-defined parameter correspondence tables, enabling better generalization and lower migration overhead when adapting to new device vendors (Section IV-G).

Performance on smaller LLMs. We also evaluate Llama3.1-8B [10] and Qwen3-8B [37], and observe two key limitations. (1) They struggle to produce well-formatted outputs, often fail to generate valid JSON during intent extraction and configuration splitting. (2) They lack sufficient understanding

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

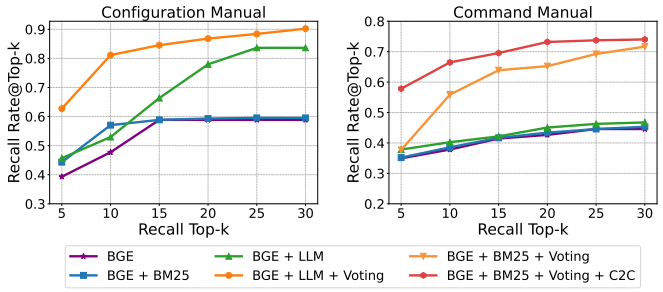


Fig. 5. Ablation study of the intent-based retrieval module.

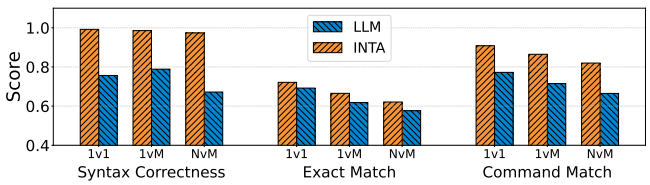


Fig. 6. Deep dive into varying levels of translation difficulty.

of configuration semantics and vendor-specific knowledge, resulting in lower translation quality. For successful cases, Llama3.1-8B achieves 55.96% syntax accuracy and 41.91% command match rate, while Qwen3-8B reaches 73.76% and 55.18%. The results show that small-scale 8B LLMs remain inadequate for configuration translation.

Case study. In Appendix D, we use a concrete example to show the step-by-step translation process of INTA.

C. Intent-Based Manual Retrieval Module

To evaluate the recall rate of the intent-based manual retrieval module, we manually annotate 409 mappings from source device configurations to target device configuration/command manuals, which serve as the test set for module evaluation. We use Qwen-Max as the default LLM base model.

We conduct an ablation study to verify the effectiveness of each module in the manual retrieval process. The results are shown in Fig. 5. The baseline method is *BGE*, which uses only the BGE embedding model for retrieval. *LLM* and *BM25* serve as preliminary filtering strategies. *Voting* denotes a voting mechanism that combines the retrieval results of multiple intents. *C2C* refers to retrieving related command manuals based on the retrieved configuration manuals.

For configuration manual retrieval, using *LLM* as a preliminary filter significantly improves the "tail-end" performance (Top-20 to Top-30), while *BM25* has almost no effect on improving the recall rate. Building on the *LLM* filter, the *Voting* mechanism further improves the "head-end" recall rate (Top-5 to Top-20), indicating that aggregating retrieval results based on fine-grained intent interpretations effectively boosts the retrieval of relevant manuals.

For command manual retrieval, neither *BM25* nor *LLM* improves retrieval accuracy effectively, suggesting that the features of command manuals are more difficult to capture. Nevertheless, we opt for the more efficient and lightweight *BM25* to reduce the candidate manual set and accelerate the subsequent dense retrieval process. The *Voting* mechanism significantly enhances overall performance, particularly improving the recall rate in the "tail-end". Meanwhile, the *C2C* mechanism further boosts the "head-end" recall rate, indicating that the configuration information contained in accurately retrieved configuration manuals is highly relevant and critical for command manual retrieval.

TABLE IV
QUALITY OF TRANSLATION REPORTS.

Model	TN	TP	FN	FP	Accuracy
GPT-4o	73	10	0	6	93.26%
Qwen-Max	64	9	0	10	87.95%
DeepSeek-V3	87	10	0	8	92.38%

D. Deep Dive

Different difficulty levels. To further evaluate the performance of INTA under varying levels of translation difficulty, we divide the dataset into three types: (1) one source command maps one target command (1v1); (2) one source command maps M target commands (1vM); (3) N source commands map M target commands (NvM). The 1vM category also includes cases where M source commands map to one target command. Fig. 6 shows the results using DeepSeek-V3 as the base model. INTA consistently outperforms LLM-only translation across all types. Notably, on the most challenging NvM subset, it achieves the largest gain in syntax correctness. These results underscore INTA's effectiveness, particularly in handling complex configuration translation scenarios.

Different input lengths. Additional experiments demonstrate that INTA maintains stable performance across different input configuration lengths within a certain range, with detailed results provided in Appendix F.

Configuration name preservation. We further analyzed INTA's translation results and observed that 96.49% of configuration names (e.g., interface or route policy) remained consistent with the source when both source and target configurations required explicit naming.

E. Human Evaluation for Translation Report

To assess the quality of the translation reports, we conduct a manual evaluation with industry experts using a Huawei NE40E router in the GNS3 emulator. We randomly select 12 reports and evaluate all report units within them. The number of report units varies by model. The evaluation metrics include *True Negative (TN)*, *True Positive (TP)*, *False Negative (FN)*, *False Positive (FP)*, and *Accuracy*. Here, we treat *incorrect configurations* as *positive* cases. The evaluation results are shown in Table IV. It shows that the LLMs provide translation reports with relatively high accuracy. They do not miss incorrect configurations (i.e., zero false negatives), but sometimes misclassify correct configurations as incorrect ones (i.e., false positives). This is mainly because LLMs are too rigorous in capturing the literal differences between configurations, lead-

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CISCO-TO-HUAWEI SWITCH SCENARIO EVALUATION.

Method	Syntax Correctness	Exact Match	Command Match
LLM (DeepSeek-V3)	0.7188	0.3742	0.6950
INTA (DeepSeek-V3)	0.9650	0.6431	0.7884

ing to misclassifying some correct configurations as incorrect ones. Nevertheless, thanks to the high recall rate for incorrect configurations, the reports still offer practical guidance for operational use.

F. Exploring Generalization Capability

To validate the effectiveness of our approach in other scenarios, we conduct an evaluation on translating configurations from Cisco to Huawei switches. We select 146 configuration cases from the Cisco configuration manual as the test set. The evaluation results are shown in Table V. In this scenario, INTA also demonstrates significant improvements over the LLM baseline. The results validate INTA's generalization capability across different scenarios.

G. Overhead

Runtime overhead. We define *runtime overhead* as the average time and monetary cost *per line* (translating one line of source device configuration). Using Qwen-Max as the base model, the full INTA pipeline takes an average of 16.12 seconds per line on an NVIDIA A100 GPU, and 30.93 seconds on an NVIDIA RTX 3090 GPU. As for LLM cost, it consumes an average of 3536 prompt tokens and 276 completion tokens per line. This corresponds to approximately $0.0015 per line with Qwen-Max, or $0.0116 with GPT-4o. Based on our practical experience and industry data, this process typically takes hours per line with high labor costs. INTA achieves substantially lower time and monetary overhead.

Migration overhead. We define *migration overhead* as the total effort required to support a new device vendor, measured by (1) lines of code (LoC) for system extension and (2) estimated human effort in person-days. The estimated migration overheads are as follows: INTA requires $\sim$100 LoC and 2 person-days per new device, involving only manual scraping of device manuals and minor parser modifications, without the need for predefined mapping rules. ConfigTrans incurs substantially higher overhead, requiring a $\sim$500 LoC and 14 person-days per configuration mode due to the need for heuristic algorithm design and command parameter mapping table construction. NAssim, although not an end-to-end system, has a similar migration overhead to INTA.

V. DISCUSSION

A. Practical Application Value

INTA significantly reduces cost and time compared to traditional methods that rely heavily on manual effort during vendor migration. It delivers reliable accuracy in generating translated configurations, with no false positives observed in translation reports across the evaluated dataset. This enables users to focus solely on flagged errors, substantially reducing the workload for human experts. Even when occasional false positives arise, the translated configurations and accompanying reports still provide valuable guidance, requiring far less effort than translating from scratch. While not yet fully autonomous, INTA already serves as a practical assistant by offering relatively reliable translations to assist human experts.

B. Limitations and Future Work

Although INTA performs well in cross-vendor configuration translation, it still has limitations and suggests future work.

Unified context tracking: INTA currently employs incremental translation to manage dependencies across configuration fragments. Future work includes developing a more robust and unified context tracking mechanism.

Hybrid verification: The INTA framework is currently using a syntax tree and LLM for verification and refinement. Future work includes formal and simulation-based verification.

Real device interaction: Future improvements also include incorporating real-time device status (e.g., port status) to enhance the practicality and accuracy of translation.

Network-wide scenarios: INTA currently focuses on translating configurations for individual devices. Future work includes incorporating topology-aware context and modeling inter-device dependencies to support network-wide scenarios.

SDN/NFV integration: Future work also includes extending INTA to support dynamic, virtualized, and service-oriented SDN/NFV scenarios for more efficient network management.

VI. RELATED WORK

NAssim [7] generates and validates configuration models from device manuals to integrate legacy devices into SDN networks. ConfigTrans [28] combines constraint solving and LLMs for configuration translation. Nissist [3] uses LLMs to clarify intent and extracts information from knowledge bases for network troubleshooting. Lumi [23] translates human intents into an abstraction layer *Nile* via machine learning, but can't generate low-level device-specific configuration commands. Nissist and Lumi primarily focus on capturing and analyzing human intent, while INTA introduces a novel approach by using intent to bridge heterogeneous configuration models. Batfish [12] uses a rule-based intermediate vendor-neutral representation for network configuration analysis, but it suffers from limited coverage and high migration overhead. CEGS [26] automates network configuration synthesis using graph neural networks (GNNs) and LLMs, but its capability and coverage rely on high-quality configuration examples. Verified Prompt Programming [27] combines GPT-4 with verifiers to generate correct router configurations.

VII. CONCLUSION

Traditional network configuration translation methods are labor-intensive due to diverse configuration models. We propose INTA, an LLM-driven intent-based framework with four key modules to address this challenge. Experiments on industry datasets show that INTA outperforms existing methods and exhibits generalization across various network scenarios, highlighting its practical value for real-world applications.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] GNS3 emulator. https://gns3.com/, 2025.

[2] Alibaba Group. Qwen-Max. https://bailian.console.aliyun.com/model-market/detail/qwen-max#/model-market/detail/qwen-max, 2025.

[3] Kaikai An, Fangkai Yang, Junting Lu, Liqun Li, Zhixing Ren, Hao Huang, Lu Wang, Pu Zhao, Yu Kang, Hua Ding, et al. Nissist: An incident mitigation copilot based on troubleshooting guides. *ECAI Demo Track*, 2024.

[4] Theophilus Benson, Aditya Akella, and David A Maltz. Unraveling the complexity of network management. In *NSDI*, pages 335–348, 2009.

[5] Amanda Bertsch, Maor Ivgi, Uri Alon, Jonathan Berant, Matthew R Gormley, and Graham Neubig. In-context learning with long-context models: An in-depth exploration. *arXiv preprint arXiv:2405.00200*, 2024.

[6] Martin Björklund. YANG - A Data Modeling Language for the Network Configuration Protocol (NETCONF). RFC 6020, October 2010.

[7] Huangxun Chen, Yukai Miao, Li Chen, Haifeng Sun, Hong Xu, Libin Liu, Gong Zhang, and Wei Wang. Software-defined network assimilation: bridging the last mile towards centralized network configuration management with NAssim. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 281–297, 2022.

[8] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.

[9] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. A survey on in-context learning. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1107–1128, 2024.

[10] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024.

[11] Rob Enns. NETCONF Configuration Protocol. RFC 4741, December 2006.

[12] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 469–483, 2015.

[13] GSMA. Migration from physical to virtual network functions: Best practices and lessons learned. https://www.gsma.com/solutions-and-impact/technologies/networks/5g/migration-from-physical-to-virtual-network-functions-best-practices-and-lessons-learned/, 2018.

[14] Joseph C Hartman and Chin Hon Tan. Equipment replacement analysis: a literature review and directions for future research. *The engineering economist*, 59(2):136–153, 2014.

[15] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024.

[16] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[17] Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A survey. In *61st Annual Meeting of the Association for Computational Linguistics, ACL 2023*, pages 1049–1065. Association for Computational Linguistics (ACL), 2023.

[18] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[19] Huawei Technologies Co., Ltd. NE40E-M2 product documentation. https://support.huawei.com/hedex/hdx.do?docid=EDOC1100331639, 2025.

[20] Huawei Technologies Co., Ltd. NE40E-M2 product documentation - command reference. https://support.huawei.com/hedex/hdx.do?docid=EDOC1100331639&id=EN-US_CLIREF_0000001759395173, 2025.

[21] Huawei Technologies Co., Ltd. NE40E-M2 product documentation - configuration. https://support.huawei.com/hedex/hdx.do?docid=EDOC1100331639&id=EN-US_TOPIC_0000001501814785, 2025.

[22] Huawei Technologies Co., Ltd. NE40E-M2 product documentation - ip address (interface view). https://support.huawei.com/hedex/hdx.do?docid=EDOC1100331639&id=EN-US_CLIREF_0000001759559309, 2025.

[23] Arthur S Jacobs, Ricardo J Pfitscher, Rafael H Ribeiro, Ronaldo A Ferreira, Lisandro Z Granville, Walter Willinger, and Sanjay G Rao. Hey, Lumi! using natural language for intent-based network management. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 625–639, 2021.

[24] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhu Chen. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060*, 2024.

[25] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[26] Jianmin Liu, Li Chen, Dan Li, and Yukai Miao. CEGS: Configuration example generalizing synthesizer. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1327–1347, 2025.

[27] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, and George Varghese. What do llms need to synthesize correct router configurations? In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, pages 189–195, 2023.

[28] Zheng Naigong, Li Fuliang, Li Ziming, Yang Yu, Hao Yimo, Liu Chenyang, and Wang Xingwei. Configtrans: Network configuration translation based on large language models and constraint solving. In *The 32nd IEEE International Conference on Network Protocols (ICNP 2024)*, 2024.

[29] Nokia. Nokia SR OS 24-7 configuration guide. https://documentation.nokia.com/sr/24-7/index.html, 2025.

[30] OpenAI. GPT-4o. https://openai.com/index/hello-gpt-4o/, 2025.

[31] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.

[32] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, November 2019. Association for Computational Linguistics.

[33] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

[34] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. *Advances in neural information processing systems*, 33:20601–20611, 2020.

[35] Philip A. Shafer. An Architecture for Network Management Using NETCONF and YANG. RFC 6244, June 2011.

[36] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.

[37] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[38] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. Exploring and unleashing the power of large language models in automated code translation. *Proceedings of the ACM on Software Engineering*, 1(FSE):1585–1608, 2024.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

APPENDIX A
CONFIGURATION PARSER DETAILS

We use the configuration parser in INTA for two purposes: (1) to extract the exact manual pages corresponding to the source configurations, which facilitates configuration division and intent extraction; (2) to verify the syntax correctness of the translated configurations and guide syntax refinement during the incremental translation process. The parser consists of two interdependent core modules: the command syntax parser and the command hierarchy parser. We construct the parser using an approach similar to that of NAssim [7].

Command syntax parser. The command syntax parser constructs a command graph based on the command manual, which serves as a matching template. The construction of the parser relies on the command template conventions described in the command manual, as illustrated in Fig. 7. These command graphs enable line-by-line parsing of commands and extraction of their syntax trees. The command graph includes three types of non-leaf nodes: `seq` (sequential nodes), `req_selector` (corresponding to `{x|y|...}`), and `opt_selector` (corresponding to `[x|y|...]`). Other conventions, such as `*` and `&` sign, are implemented as features of the nodes. It also contains two types of leaf nodes: keyword nodes and parameter nodes. In addition, there are two functional node types: the `end` node (indicating the end of a command) and the `pass` node (serving as a placeholder in `opt_selector`). For a given command template (e.g., the format shown in Fig. 8), we traverse the command format specification from left to right to construct the command graph. The command graph is built using a recursive structure, starting with an outermost `seq` node. When encountering an optional item, a corresponding `req_selector` or `opt_selector` node is created. The constructed command graph is then used to match actual configuration commands, associating elements in the configuration (such as keywords and parameters) with corresponding elements in the command graph.

Command hierarchy parser. Based on the command syntax parser, we build a command hierarchy parser to analyze the hierarchical view structure of configuration files. The command hierarchy parser relies on the device's command hierarchy file, namely the VDM in NAssim [7]. A VDM file is a JSON file that specifies, for each command, its type, CLI definition, associated view, and all subcommands (children) available within the view it enters. For a complete configuration file, starting from the root node of the VDM (e.g., `system-view` command in Huawei devices' system view), the hierarchy parser uses the command syntax parser to parse the configuration file line by line, match commands in the VDM, and record the corresponding view hierarchy transitions. The parsed output not only contains the syntax matching results for each command but also includes the hierarchical view context to which each command belongs.

Example of adapting to a new vendor. To illustrate the migration effort required when adapting the parser to a new vendor, we provide a concrete example using Huawei NE40E.

Command Conventions

The command conventions that may be found in this document are defined as follows.

Convention	Description
Boldface	The keywords of a command line are in **boldface**.
Italic	Command arguments are in *italics*.
[ ]	Items (keywords or arguments) in brackets [ ] are optional.
{ x | y | ... }	Optional items are grouped in braces and separated by vertical bars. One item is selected.
[x | y | ...]	Optional items are grouped in brackets and separated by vertical bars. One item is selected or no item is selected.
{ x | y | ... }*	Optional items are grouped in braces and separated by vertical bars. A minimum of one item or a maximum of all items can be selected.
[x | y | ...]*	Optional items are grouped in brackets and separated by vertical bars. Several items or no item can be selected.
&<1-n>	The parameter before the & sign can be repeated 1 to n times.
#	A line starting with the # sign is comments.

Fig. 7. Command conventions of Huawei command manuals.

The *Command Syntax Parser* is decoupled from vendor-specific details and only relies on a unified command template convention (Fig. 7). Therefore, as long as the target vendor's documentation can be transformed into this convention, the syntax parser remains fully reusable without any modification.

The *Command Hierarchy Parser* is vendor-specific and reflects differences in configuration models. Taking Huawei as an example, the main modification lies in supporting a large number of interface views. In the Huawei NE40E router, simply recognizing the `interface` command does not lead to a single fixed view because there are 109 possible interface view types. Therefore, the parser must recognize the exact interface type (e.g., GE, Loopback, VLANIF) to determine the correct sub-view to enter. Moreover, the parser must also distinguish between sub-interface views and other variants. As a result, the main adaptation effort for Huawei NE40E lies in handling these diverse interface views.

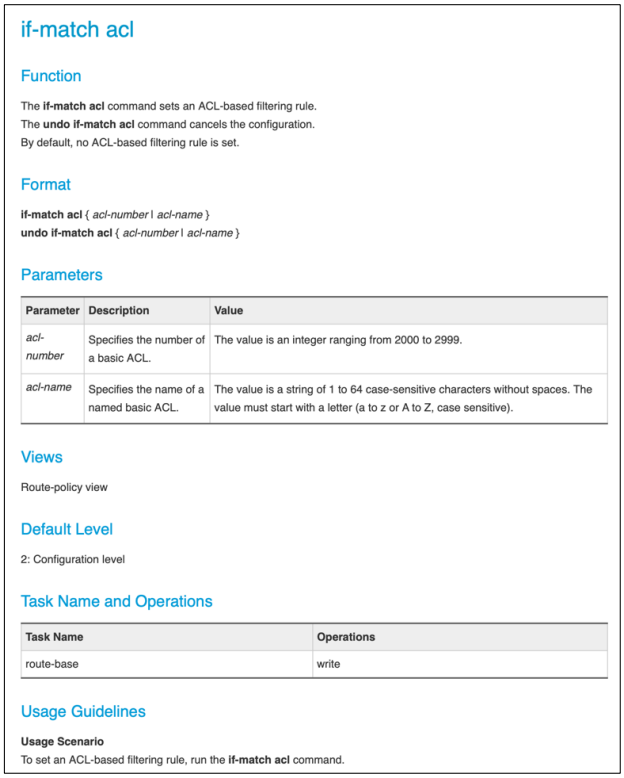
APPENDIX B
MANUAL EXAMPLES

We use two types of manuals in our work: *Command Manuals* and *Configuration Manuals*. We use Huawei NE40E Router manuals as examples to illustrate them.

Command manuals. Contains syntax definitions and functional descriptions of all commands of the device, mainly used for constructing the configuration syntax tree in the parser. Command manuals are also retrieved to enhance the translation process. A command manual example is shown in Fig. 8.

Configuration manuals. Contains configuration steps (command sequence) required to implement a certain function. We use extracted intents to retrieve the corresponding configuration manuals, which are used to generate the configuration. A configuration manual example is shown in Fig. 9.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.



if-match acl

Function

The **if-match acl** command sets an ACL-based filtering rule.
The **undo if-match acl** command cancels the configuration.
By default, no ACL-based filtering rule is set.

Format

if-match acl { *acl-number* | *acl-name* }
undo if-match acl { *acl-number* | *acl-name* }

Parameters

Parameter	Description	Value
acl-number	Specifies the number of a basic ACL.	The value is an integer ranging from 2000 to 2999.
acl-name	Specifies the name of a named basic ACL.	The value is a string of 1 to 64 case-sensitive characters without spaces. The value must start with a letter (a to z or A to Z, case sensitive).

Views

Route-policy view

Default Level

2: Configuration level

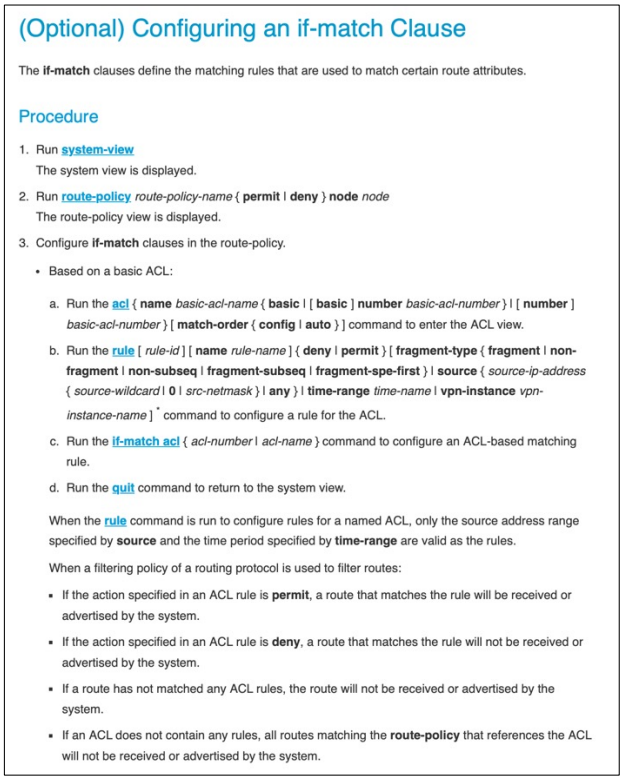
Task Name and Operations

Task Name	Operations
route-base	write

Usage Guidelines

Usage Scenario
To set an ACL-based filtering rule, run the **if-match acl** command.

Fig. 8. Command manual example.



(Optional) Configuring an if-match Clause

The **if-match** clauses define the matching rules that are used to match certain route attributes.

Procedure

1. Run **system-view**
   The system view is displayed.
2. Run **route-policy** *route-policy-name* { **permit** | **deny** } **node** *node*
   The route-policy view is displayed.
3. Configure **if-match** clauses in the route-policy.
   - Based on a basic ACL:
     a. Run the **acl** { **name** *basic-acl-name* { **basic** | [**basic**] **number** *basic-acl-number* } | [**number**] *basic-acl-number* } [**match-order** { **config** | **auto** }] command to enter the ACL view.
     b. Run the **rule** [*rule-id*] [**name** *rule-name*] { **deny** | **permit** } [**fragment-type** { **fragment** | **non-fragment** | **non-subseq** | **fragment-subseq** | **fragment-spe-first** } | **source** { *source-ip-address* { *source-wildcard* | **0** | *src-netmask* } | **any** } | **time-range** *time-name* | **vpn-instance** *vpn-instance-name*] * command to configure a rule for the ACL.
     c. Run the **if-match acl** { *acl-number* | *acl-name* } command to configure an ACL-based matching rule.
     d. Run the **quit** command to return to the system view.

     When the **rule** command is run to configure rules for a named ACL, only the source address range specified by **source** and the time period specified by **time-range** are valid as the rules.

     When a filtering policy of a routing protocol is used to filter routes:
     - If the action specified in an ACL rule is **permit**, a route that matches the rule will be received or advertised by the system.
     - If the action specified in an ACL rule is **deny**, a route that matches the rule will not be received or advertised by the system.
     - If a route has not matched any ACL rules, the route will not be received or advertised by the system.
     - If an ACL does not contain any rules, all routes matching the **route-policy** that references the ACL will not be received or advertised by the system.

Fig. 9. Configuration manual example.

```
# TASK DESCRIPTION
Your task is to analyze the following {src_device} configuration,
divide it into fragments based on intents and behaviors and give a
general description of each fragment, together with a detailed
description of each sub module in the fragment. To assist you,
corresponding {src_device} manuals are provided in the ## MANUALS
section. We also provide you with an example.

## OUTPUT REQUIREMENTS
Store the fragments in a SINGLE list of JSON structure with:
- **"Fragment"**: The original Nokia configuration segment.
- **"Intent"**: General intent description of the fragment.
- **"Details"**: Detailed intent description of each sub-module.

# EXAMPLE
(intent examples)

# TEMPLATE
```json
[
  {
    "Fragment": "...",
    "Intent":  "...",
    "Details": {
      "xxx(cmd)": {
        "Intent": "...",
        "Parameters": {...}
      },
    },
  },
...
]
```

YOUR TASK
CONFIGURATION
{configuration}

MANUALS
{manuals}

RESULT
```json Your answer here (a single list of all fragments)```
```

Fig. 10. Example prompt template for intent extraction.

## APPENDIX C
## DETAILS AND PROMPT TEMPLATES FOR EACH MODULE

In this appendix, we provide some details and prompt templates used in each module of INTA.

**System prompt.** The unified system prompt used in INTA is: *You are a very helpful assistant with great expertise in network operations and maintenance.*

**Configuration division and intent extraction.** The prompt template for intent extraction is shown in Fig. 10. We provide LLM with a JSON format instruction, together with an output template indicating the expected output format. An example fragment of configuration division and intent extraction result is shown in Fig. 11.

**LLM corpus filter.** The prompt template for LLM corpus filter is shown in Fig. 12. The output of the LLM corpus filter is a list of command/configuration manual paths for further retrieval.

**Incremental translation.** The prompt template of incremental translation and syntax refinement is shown in Fig. 13 and Fig. 14, respectively.

**Verification.** Syntax verification uses the configuration tree (Appendix A) to analyze the syntax errors. Semantic verification and correction are both implemented with LLMs. The prompt template for semantic verification and refinement is

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.
```

```
"Fragment": "prefix-list \"SYSTEM\"\n prefix xx.x.xx.xx/32 prefix-
length-range 16-32",
"Intent": "Define a prefix list to match IP prefixes within a
specified prefix length range",
"Details": {
  "prefix-list": {
    "Intent": "Create a prefix list that matches IP prefixes",
    "Parameter": {
      "name": "SYSTEM"
    }
  },
  "prefix": {
    "Intent": "Specify the exact IP prefix for the prefix list entry",
    "Parameter": {
      "ip-prefix": "xx.x.xx.xx/32",
      "prefix-length-range": "16-32"
    }
  }
}
```

Fig. 11. Example fragment of configuration division and intent extraction result.

```
## TASK DESCRIPTION
We are translating the provided {src_device} configuration fragment
(### SOURCE DEVICE CONFIGURATION FRAGMENT) into {tgt_device}
configuration. Your task is to help me read the manual catalogue and
find the most relevant entries for configuration translation.
To complete this task, you need to:
1. Traverse the ### {tgt_device} CONFIGURATION MANUAL LIST.
2. Select the most relevant manual directory(ies) to enter to guide
the translation.

### GUIDELINES
- You are provided with some background information of ### SOURCE
DEVICE MANUALS and ### SOURCE DEVICE FULL CONFIGURATION to help you
understand the fragment.
- Your selection must be the EXACT directory name(s) with FULL path
from the manual list.
- Provide your selection in a list format, e.g., `["parent1/child1",
"parent2/child2"]`.

### {src_device} CONFIGURATION FRAGMENT
{Source_Config_Fragment}

### {src_device} FULL CONFIGURATION
{Source_Config}

### {src_device} MANUALS
{Source_Config_Manual}

### {tgt_device} CONFIGURATION MANUAL LIST
{Target_Config_Manual_list}

### OUTPUT FORMAT
```plaintext
["your selection here", ...]
```
Take it step by step.
"""
```

Fig. 12. Example prompt template for LLM corpus filter.

shown in Fig. 16 and Fig. 17, respectively. Fig. 15 shows the semantic report template used in Fig. 16.

APPENDIX D
CASE STUDY

We provide a case study of the configuration translation process with INTA in Fig. 18. To translate the fragment in NOKIA Source Configuration, the intent extraction module extracts the intent from the fragment. Then, the target manual retrieval module uses the extracted intents to retrieve the corresponding configuration and command manual pages. The incremental

```
## TASK DESCRIPTION
Your task is to translate a given {src_device} configuration fragment
in ## {src_device} CONFIGURATION FRAGMENT into a {tgt_device}
configuration.

## REQUIREMENTS
 - Use the ### {src_device} MANUALS section to fully understand the
source configuration commands and structure.
 - Use the ### {tgt_device} MANUALS section to translate source
commands into target equivalents, following target syntax and
conventions.
 - Incorporate the target configuration fragment seamlessly into the
### PRECEDING TRANSLATION section.

## GUIDELINES
Target device command conventions:
 - [ ]: Items (keywords or arguments) in brackets [ ] are optional.
 - {{ x | y | ... }}: Optional items are grouped in braces and
separated by vertical bars. One item is selected.
 - [ x | y | ... ]: Optional items are grouped in brackets and
separated by vertical bars. One item is selected or no item is
selected.

## {src_device} MANUALS
{src_manuals}

## {tgt_device} CONFIGURATION MANUALS
{tgt_config_manuals}

## {tgt_device} COMMAND MANUALS
{tgt_cmd_manuals}

## PRECEDING {tgt_device} CONFIGURATION TRANSLATION RESULT
{preceding_translation}

## {src_device} CONFIGURATION FRAGMENT
{config_fragment}

## RESULT
```plaintext (COMPLETE {tgt_device} configuration here)```
```

Fig. 13. Example prompt template for incremental translation.

```
Thank you for your translation. We have verified your translation in
the {tgt_device} configuration tree and identified mismatches listed
in the ## MISMATCHED LINES section below.

### Task:
Your goal is to refine the translation to resolve these mismatches.
- Carefully review the mismatched lines.
- Revise your translation to address the mismatches.
- Provide the complete updated translation, including the refined
sections. Do not submit only the changes; the entire configuration
must be included.
- Do not loss or add any functions during the refinement process.
- Keep the indentation to represent view structure

### Mismatched Lines:
{mismatched_lines}

### Output Format:
Provide the full refined translation within the following template:
```plaintext (Your complete revised translation here) ```
```

Fig. 14. Example prompt template for syntax refinement.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

```
[
    // if there are corresponding fragments
    {{
        "{src_device}": "...",
        "{tgt_device}": "...",
        "Equivalent": "True/False",
        "Comments": "..."
    }},
    // if there are no corresponding fragments in translated
{tgt_device} configuration
    {{
        "{src_device}": "...",
        "{tgt_device}": null,
        "Equivalent": "False",
        "Comments": "Missing fragment in {tgt_device} configuration"
    }},
    // if there are additional fragments in translated {tgt_device}
configuration"
    {{
        "{src_device}": null,
        "{tgt_device}": "...",
        "Equivalent": "False",
        "Comments": "Additional fragment in {tgt_device}
configuration"
    }},
    ...
]
```

Fig. 15. Report template for semantic verification.

```
## TASK DESCRIPTION
You are required to evaluate the following {tgt_device} configuration
based on the provided {src_device} configuration and manuals. The
syntax, semantics, and functionality of the translated configuration
should be equal.
Compare the configurations in semantic units, each unit being a
fragment of the configuration.
Provide a detailed analysis of the translation quality, unit by unit,
and give a critique in the following format:
{semantic_report_template}
To help you understand the configuration, the corresponding
{src_device} manuals are provided in the ### {src_device} MANUALS
section. The corresponding {tgt_device} manuals are provided in the
### {tgt_device} MANUALS section.

## {src_device} MANUALS
{src_manuals}

## {tgt_device} MANUALS
{tgt_manuals}

## INITIAL {src_device} CONFIGURATION
{configuration}

## TRANSLATED {tgt_device} CONFIGURATION
{translation}

## RESULT
Give a full JSON list of all the fragments at the end of your answer.
```json (your full answer) here```
```

Fig. 16. Example prompt template for semantic verification.

translation module uses the retrieved manual pages and the source configuration fragment to generate the translated configuration fragment. The target command `ospf enable` is translated successfully with the correct view (interface view). This is because both the command and the view information are included in the manual page. In contrast, the translation result of GPT-4o not only misses this command but also provides incorrect view information.

```
## TASK DESCRIPTION
Your task is to refine a {tgt_device} configuration based on the
provided {src_device} configuration, relevant manuals, and identified
semantic warnings.
## REQUIREMENTS
1. Address the semantic warnings listed in the ## SEMANTIC WARNINGS
section.
2. Ensure the refined {tgt_device} configuration retains the same
functionality and semantics as the original {src_device} configuration.
3. Verify all changes align with the manuals provided in the ###
{src_device} MANUALS and ### {tgt_device} MANUALS sections if the
corresponding manual exists.
4. Complete and Clean Output
    - Provide the FULL refined {tgt_device} configuration in the
RESULT section.
    - Your refined configuration should be clean, without any comments.
    - Keep indentation to represent view structure.

## {src_device} MANUALS
{src_manuals}

## {tgt_device} MANUALS
{tgt_manuals}

## {src_device} CONFIGURATION
{src_configuration}

## {tgt_device} CONFIGURATION
{tgt_configuration}

## SEMANTIC WARNINGS
{semantic_errors}

## RESULT
```plaintext (complete refined {tgt_device} configuration here)```
```

Fig. 17. Example prompt template for semantic refinement.

TABLE VI
MODEL COMPARISON ON CONFIGURATION AND COMMAND MANUAL RECALL RATE@TOP-30.

Metric	Qwen-Max	GPT-4o	DeepSeek-V3
Configuration Manual	0.9023	0.8617	0.8662
Command Manual	0.7402	0.6578	0.7470

APPENDIX E
PARAMETER AND MODEL SELECTION FOR INTENT-BASED MANUAL RETRIEVAL MODULE

Parameter selections. In the intent-based manual retrieval module, the number of manuals retrieved for a single entry of intent description, denoted as top-k, is a key parameter. We analyze the effect of this parameter on the overall recall rate, as shown in Fig. 19. In the figure, the choice of top-k has some impact on the overall Recall Rate@Top-k. The best performance is achieved when k=15. In the actual translation process, considering the trade-off between LLM usage cost (input context length) and accuracy, we select the top 20 final retrieved manuals as input to the incremental translation module as their recall rate is close to that of the top 30.

Performance of different LLMs and embedding models. We compare the performance of different LLMs in terms of Recall Rate@Top-30 on configuration and command manuals, as shown in Table VI. All three models perform well across both types of manuals, with minor variations. We adopt the BGE-M3 model (with 568M parameters) as our default embedding model, striking a balance between efficiency and

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.

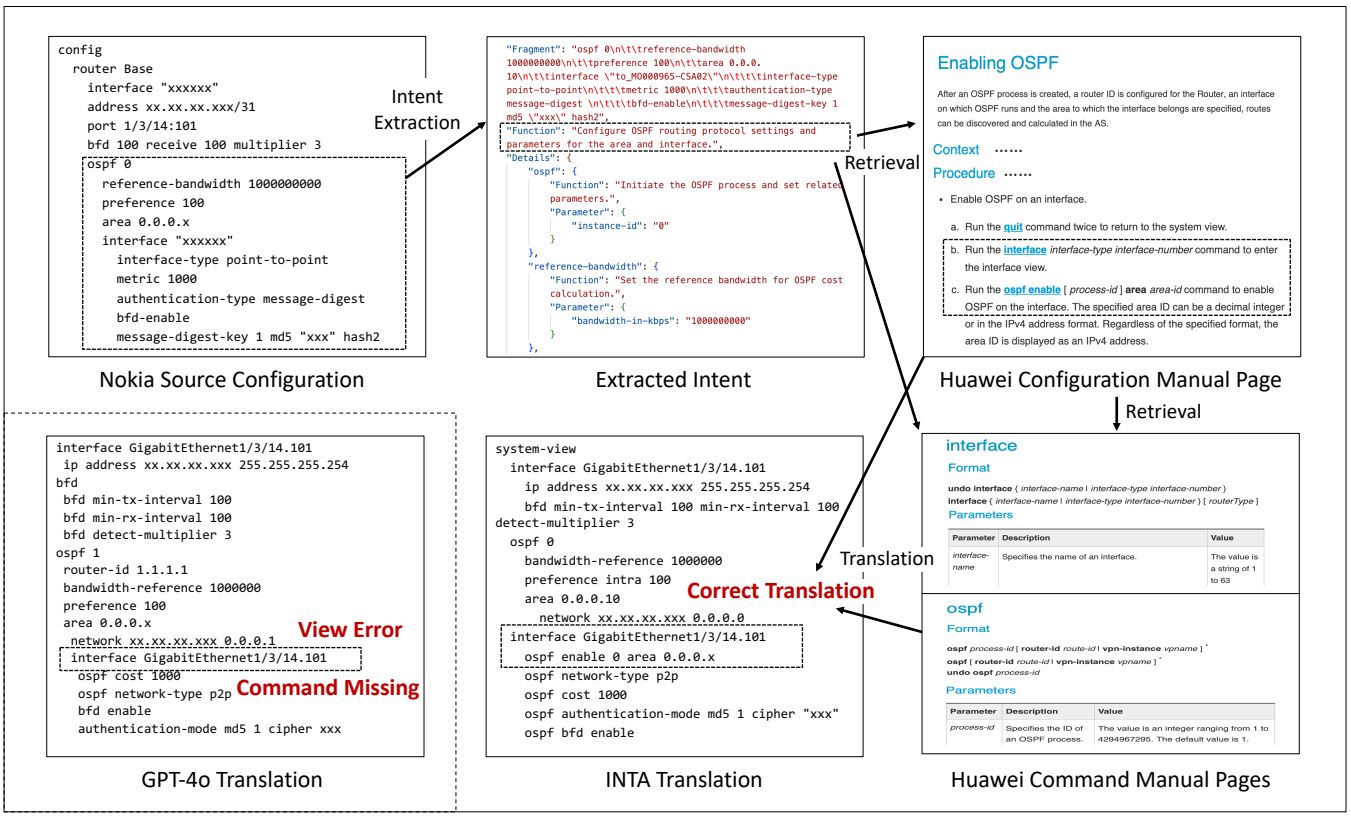


Fig. 18. A case study of the configuration translation process with INTA.

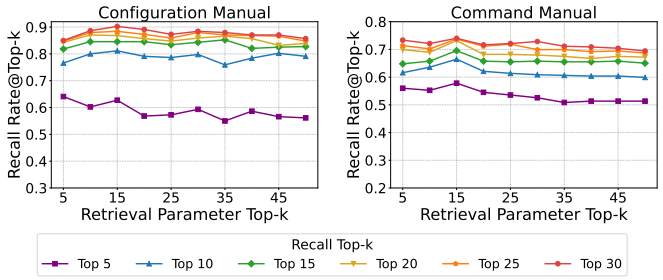


Fig. 19. Relationship between single entry retrieval parameter Top-k and overall Recall Rate@top-k.

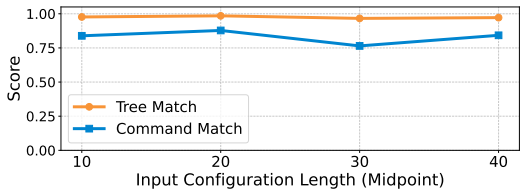


Fig. 20. INTA performance across varying configuration lengths (INTA with DeepSeek-V3). Each x-axis value represents the midpoint of a length interval, where the source device configurations fall within the range $[x-5,\ x+5)$.

performance. For comparison, we also evaluated gte-Qwen2-1.5B-Instruct (with 1.5B parameters). However, the latter achieved Recall Rate@Top-30 of 79.63% and 71.76% on the configuration and command manuals, respectively, both slightly lower than those of the BGE model. This suggests that BGE-M3 is better suited to our task.

Appendix F
Performance Across Configuration Lengths

To analyze INTA's performance variation with input length, we group source configurations by length and evaluate performance across these groups. Due to data availability, our dataset includes source device configuration files ranging from 10 to 40 lines, each representing a semantically complete unit. As shown in Fig. 20, INTA maintains stable performance across this range, with both Tree Match and Command Match metrics showing no degradation as length increases. This range also reflects practical translation units, as network engineers often translate modular blocks in a single pass. For example, core OSPF and BGP configurations typically span only a few dozen lines, while ACL policies may reach several hundred. Although our experiments are constrained by data availability, INTA's architecture is designed with the capacity to scale to much longer inputs. Its fragment-based architecture enables incremental translation and scales naturally, with the maximum input length primarily bounded by the LLM's context window. This positions INTA to handle increasingly complex configurations as model capacity continues to grow.

Authorized licensed use limited to: Tsinghua University. Downloaded on August 26,2026 at 02:10:34 UTC from IEEE Xplore. Restrictions apply.